

附录 程序源代码（按第 4 章模块整理）

以下代码依据项目实际源码整理，按照第 4 章“系统详细设计及实现”的模块顺序选取核心方法，省略实体类、Repository 接口、Getter/Setter 和样式代码，仅保留与业务流程、推荐算法、AI 处理、广告分发和数据统计直接相关的关键代码。

A.1 对应 4.1 用户认证与内容互动模块的核心代码

本模块保留用户注册登录、内容发布、互动行为写入等核心代码，用于说明用户侧基础业务闭环。

（1）用户注册逻辑

代码来源：backend/src/main/java/com/example/rec/service/AuthService.java（第 23-39 行）

```
public User register(User user) {
    if (userRepository.findByUsername(user.getUsername()).isPresent()) {
        throw new RuntimeException("Username already exists");
    }
    // TODO: 在生产环境中，密码必须加密（例如使用 BCrypt）
    user.setRole("USER");

    // 自动生成随机头像（使用 DiceBear API）
    if (user.getAvatarUrl() == null) {
        user.setAvatarUrl("https://api.dicebear.com/7.x/avataaars/svg?seed=" + user.getUsername());
    }
    // 自动生成 Handle（例如 @john）
    if (user.getHandle() == null) {
        user.setHandle("@ " + user.getUsername().toLowerCase().replace(" ", ""));
    }
    return userRepository.save(user);
}
```

（2）用户登录与封禁校验逻辑

代码来源：backend/src/main/java/com/example/rec/service/AuthService.java（第 45-59 行）

```
public User login(String username, String password) {
    Optional<User> userOpt = userRepository.findByUsername(username);
    if (userOpt.isPresent()) {
        User user = userOpt.get();
        // 检查封禁状态
        if (Boolean.TRUE.equals(user.getBanned())) {
            throw new RuntimeException("Account has been banned");
        }
        // TODO: 生产环境应比对加密后的密码
        if (user.getPassword().equals(password)) {
            return user;
        }
    }
    throw new RuntimeException("Invalid credentials");
}
```

（3）内容发布与话题标签解析逻辑

代码来源：backend/src/main/java/com/example/rec/service/ContentService.java（第 101-140 行）

```
public Content publish(Long authorId, String text, String imageUrl) {
    // 模拟：创建作者对象（在真实场景应查询 User 表）
    com.example.rec.model.User author = new com.example.rec.model.User();
    author.setId(authorId);

    Content content = new Content();
    content.setAuthor(author);
    content.setContent(text);
    content.setImageUrl(imageUrl); // 设置图片 URL（如有）
    content.setTitle("Tweet");
    content.setCategory("Life");

    // Parse Hashtags（显式 #tag）
```

```

java.util.regex.Pattern pattern = java.util.regex.Pattern.compile("#([a-zA-Z0-9_\\u4e00-\\u9fa5]+)");
java.util.regex.Matcher matcher = pattern.matcher(text);
while (matcher.find()) {
    String tagName = matcher.group(1);
    com.example.rec.model.Tag tag = tagRepository.findByName(tagName)
        .orElseGet(() -> tagRepository.save(new com.example.rec.model.Tag(tagName)));
    content.getTags().add(tag);
}

Content saved = contentRepository.save(content);

// AI 自动语义打标 (异步, 不阻塞发帖)
try {
    new Thread(() -> {
        try {
            aiTaggingService.tagContent(saved);
        } catch (Exception e) {
            System.err.println("AI auto-tagging failed for content #" + saved.getId() + ": " +
                e.getMessage());
        }
    }).start();
} catch (Exception e) {
    // AI 打标失败不影响正常发帖
    System.err.println("Failed to start AI tagging thread: " + e.getMessage());
}

return saved;
}

```

(4) 点赞行为写入与计数更新逻辑

代码来源: backend/src/main/java/com/example/rec/service/BehaviorService.java (第 50-75 行)

```

@Transactional
public void likeContent(Long userId, Long contentId) {
    List<Behavior> existingLike = behaviorRepository.findByUserIdAndContentIdAndType(userId, contentId, "LIKE");
    if (!existingLike.isEmpty()) return;

    Content content = contentRepository.findById(contentId)
        .orElseThrow(() -> new RuntimeException("Content not found"));

    List<Behavior> existingDislike = behaviorRepository.findByUserIdAndContentIdAndType(userId, contentId,
        "DISLIKE");
    if (!existingDislike.isEmpty()) {
        behaviorRepository.deleteAll(existingDislike);
        content.setDislikeCount(Math.max(0, (content.getDislikeCount() != null ? content.getDislikeCount() : 0)
            - 1));
    }

    Behavior behavior = new Behavior();
    behavior.setUserId(userId);
    behavior.setContentId(contentId);
    behavior.setType("LIKE");
    behaviorRepository.save(behavior);

    content.setLikeCount(content.getLikeCount() + 1);
    contentRepository.save(content);

    notificationService.createNotification(content.getAuthor().getId(), userId, "LIKE", contentId);
    strategyManager.invalidateAiCache(userId);
}

```

(5) 浏览行为采集逻辑

代码来源: backend/src/main/java/com/example/rec/service/BehaviorService.java (第 77-102 行)

```

@Transactional
public void recordView(Long userId, Long contentId, Integer duration) {
    if (userId == null || contentId == null) return;

    List<Behavior> existing = behaviorRepository.findByUserIdAndContentIdAndType(userId, contentId, "VIEW");
    if (!existing.isEmpty()) {
        Behavior last = existing.get(existing.size() - 1);
        if (duration != null && (last.getDuration() == null || duration > last.getDuration())) {

```

```
        last.setDuration(duration);
        behaviorRepository.save(last);
    }
    return;
}

Behavior behavior = new Behavior();
behavior.setUserId(userId);
behavior.setContentId(contentId);
behavior.setType("VIEW");
behavior.setDuration(duration);
behaviorRepository.save(behavior);

contentRepository.findById(contentId).ifPresent(content -> {
    content.setViewCount((content.getViewCount() != null ? content.getViewCount() : 0) + 1);
    contentRepository.save(content);
});
}
```

A.2 对应 4.2 推荐排序与用户画像模块的核心代码

本模块保留推荐候选池构建、混合推荐评分、近期兴趣序列和对比指标计算等代码。

(1) 推荐流入口与策略调用逻辑

代码来源: backend/src/main/java/com/example/rec/service/RecommendationService.java (第 99-117 行)

```
public List<com.example.rec.dto.ContentWithScore> getRecommendedFeedWithScore(Long userId) {
    List<Content> finalCandidates = buildCandidatePool(userId);

    List<com.example.rec.dto.ContentWithScore> result;
    Map<String, Double> customWeights = loadUserCustomWeights(userId);
    if (customWeights != null) {
        result = strategyManager.recommendWithScore(userId, finalCandidates, customWeights);
    } else {
        result = strategyManager.recommendWithScore(userId, finalCandidates);
    }

    if (userId != null && result != null && !result.isEmpty()) {
        List<Content> contents = result.stream()
            .map(com.example.rec.dto.ContentWithScore::getContent)
            .collect(Collectors.toList());
        fillIsLiked(contents, userId);
    }
    return result;
}
```

(2) 候选内容池构建逻辑

代码来源: backend/src/main/java/com/example/rec/service/RecommendationService.java (第 273-339 行)

```
private List<Content> buildCandidatePoolWithStats(Long userId, PipelineStats stats) {
    List<Content> finalCandidates = new ArrayList<>();
    Set<Long> seenIds = new HashSet<>();

    // === NEGATIVE SIGNAL FILTERING ===
    Set<Long> blockedAuthorIds = new HashSet<>();
    Set<Long> hiddenContentIds = new HashSet<>();
    if (userId != null) {
        blockedAuthorIds = negativeSignalService.getBlockedAuthorIds(userId);
        hiddenContentIds = negativeSignalService.getHiddenContentIds(userId);
    }

    // === SOURCE 1: In-Network (Thunder) ===
    List<Content> inNetworkCandidates = new ArrayList<>();
    if (userId != null) {
        List<Long> followingIds = relationService.getFollowingIds(userId);
        if (!followingIds.isEmpty()) {
            inNetworkCandidates = contentRepository.findByAuthorIdInOrderByCreatedAtDesc(followingIds);
        }
    }

    Set<Long> sourceSeenIds = new HashSet<>();
    int inNetworkCount = 0;
    for (Content c : inNetworkCandidates) {
        if (c.getParentContent() != null) continue; // 跳过回复
        if (sourceSeenIds.add(c.getId())) {
            inNetworkCount++;
        }
        if (!seenIds.contains(c.getId()) && !hiddenContentIds.contains(c.getId())) {
            if (c.getAuthor() == null || !blockedAuthorIds.contains(c.getAuthor().getId())) {
                c.setNetworkSource("IN_NETWORK");
                finalCandidates.add(c);
                seenIds.add(c.getId());
            }
        }
    }

    // === SOURCE 2: Out-of-Network (Phoenix) ===
    List<Content> outOfNetworkCandidates;
    if (userId != null) {
        outOfNetworkCandidates = contentRepository.findAllByAuthorIdNot(userId);
    } else {
        outOfNetworkCandidates = contentRepository.findAll();
    }
}
```

```

    }

    int outNetworkCount = 0;
    for (Content c : outOfNetworkCandidates) {
        if (c.getParentContent() != null) continue; // 跳过回复
        if (!sourceSeenIds.add(c.getId())) continue;
        outNetworkCount++;
        if (!seenIds.contains(c.getId()) && !hiddenContentIds.contains(c.getId())) {
            if (c.getAuthor() == null || !blockedAuthorIds.contains(c.getAuthor().getId())) {
                c.setNetworkSource("OUT_OF_NETWORK");
                finalCandidates.add(c);
                seenIds.add(c.getId());
            }
        }
    }
}

// 填充统计数据
if (stats != null) {
    stats.setInNetworkCount(inNetworkCount);
    stats.setOutNetworkCount(outNetworkCount);
}

return finalCandidates;
}

```

(3) 混合推荐排序主流程

代码来源: backend/src/main/java/com/example/rec/service/HybridRecommendationStrategy.java (第 164-214 行)

```

public List<ContentWithScore> recommendWithScore(Long userId, List<Content> candidates, Map<String, Double>
    manualWeights) {
    BehaviorProfile profile = userId != null
        ? behaviorProfileService.buildProfile(userId) : new BehaviorProfile();
    DynamicWeights dw = userId != null
        ? behaviorProfileService.computeDynamicWeights(userId) : new DynamicWeights();

    if (manualWeights != null && !manualWeights.isEmpty()) {
        applyManualOverrides(dw, manualWeights);
    }

    List<String> userInterests = getUserInterests(userId);
    Map<String, Double> tmpTfIdf = Collections.emptyMap();
    Map<String, Double> tmpIdf = Collections.emptyMap();
    if (userId != null) {
        try {
            tmpTfIdf = tfIdfService.getUserProfileVector(userId);
            if (!tmpTfIdf.isEmpty()) tmpIdf = tfIdfService.buildGlobalIdf();
        } catch (Exception ignored) {}
    }
    final Map<String, Double> userTfIdf = tmpTfIdf;
    final Map<String, Double> globalIdf = tmpIdf;

    Map<String, Double> weightsMap = dynamicWeightsToMap(dw);
    Set<Long> cfRecommendedIds = getCfRecommendedIds(userId);
    Set<Long> dislikedIds = getDislikedContentIds(userId);
    Set<Long> likedIds = getLikedContentIds(userId);
    Set<Long> viewedIds = getViewedContentIds(userId);

    List<ScoredContentWithDetails> scoredList = candidates.stream()
        .map(c -> calculateScoreWithDetails(c, userInterests, userTfIdf, globalIdf, profile, dw, weightsMap,
            cfRecommendedIds, dislikedIds, likedIds, viewedIds))
        .collect(Collectors.toList());

    applyAuthorDiversityPenaltyWithDetails(scoredList);

    Random jitterRandom = new Random();
    double explorationRange = dw.explorationFactor;
    for (ScoredContentWithDetails sc : scoredList) {
        double factor = (1.0 - explorationRange) + (jitterRandom.nextDouble() * explorationRange * 2);
        double jitteredScore = sc.getFinalScore() * factor;
        sc.getBreakdown().setFinalScore(Math.round(jitteredScore * 100.0) / 100.0);
    }

    scoredList.sort(Comparator.comparingDouble(ScoredContentWithDetails::getFinalScore).reversed());
}

```

```

List<ContentWithScore> result = new ArrayList<>();
int rank = 1;
for (ScoredContentWithDetails sc : scoredList) {
    result.add(new ContentWithScore(sc.getContent(), sc.getBreakdown(), rank++));
}
return result;
}

```

(4) 推荐评分明细计算逻辑

代码来源: backend/src/main/java/com/example/rec/service/HybridRecommendationStrategy.java (第 281-416 行)

```

private ScoredContentWithDetails calculateScoreWithDetails(Content content,
    List<String> userInterests, Map<String, Double> userTfIdf, Map<String, Double> idf,
    BehaviorProfile profile, DynamicWeights dw, Map<String, Double> weightsMap,
    Set<Long> cfRecommendedIds, Set<Long> dislikedIds,
    Set<Long> likedIds, Set<Long> viewedIds) {

    ScoreBreakdown bd = new ScoreBreakdown();
    List<String> reasons = new ArrayList<>();

    int likeCount = val(content.getLikeCount());
    int commentCount = val(content.getCommentCount());
    int viewCount = val(content.getViewCount());
    int repostCount = val(content.getRepostCount());
    bd.setLikeCount(likeCount);
    bd.setCommentCount(commentCount);
    bd.setViewCount(viewCount);
    bd.setRepostCount(repostCount);

    // 基础互动分
    double engagement = likeCount * dw.wLike + commentCount * dw.wReply
        + viewCount * WEIGHT_VIEW + repostCount * dw.wRepost;
    bd.setBaseEngagement(round(engagement));

    double engagementRate = viewCount > 0
        ? (double)(likeCount + commentCount + repostCount) / viewCount : 0;
    bd.setEngagementRate(round3(engagementRate));
    engagement *= (1.0 + engagementRate * ENGAGEMENT_RATE_WEIGHT);

    boolean inNetwork = "IN_NETWORK".equals(content.getNetworkSource());
    bd.setInNetwork(inNetwork);
    if (inNetwork) {
        engagement *= WEIGHT_IN_NETWORK_BOOST;
        reasons.add("来自你关注的人");
    }

    // 话题亲和度
    double topicAffinity = computeTopicAffinity(content, profile) * dw.wTopicAffinity;
    bd.setTopicAffinityBoost(round(topicAffinity));
    if (topicAffinity > 10) {
        List<String> matched = getMatchedTopics(content, profile);
        if (!matched.isEmpty()) reasons.add("你常看: " + String.join(", ", matched));
    }

    // 作者亲密度
    double authorAffinity = computeAuthorAffinity(content, profile) * dw.wAuthorAffinity;
    bd.setAuthorAffinityBoost(round(authorAffinity));
    if (authorAffinity > 10 && content.getAuthor() != null) {
        reasons.add("你常互动的作者: " + content.getAuthor().getUsername());
    }

    // 标签匹配
    double personalization = 0;
    StringBuilder matchedTags = new StringBuilder();
    if (userInterests != null && !userInterests.isEmpty() && content.getTags() != null) {
        for (var tag : content.getTags()) {
            if (userInterests.contains(tag.getName())) {
                personalization += dw.wTopicAffinity * 0.5;
                if (matchedTags.length() > 0) matchedTags.append(", ");
                matchedTags.append(tag.getName());
            }
        }
    }
}

```

```

    }
    bd.setPersonalizationBoost(round(personalization));
    bd.setMatchedTags(matchedTags.toString());

    // TF-IDF 内容相似度
    double similarity = 0;
    if (userTfIdf != null && !userTfIdf.isEmpty() && idf != null && !idf.isEmpty()) {
        try { similarity = tfIdfService.getContentSimilarityScore(content, userTfIdf, idf) * dw.wSimilarity; }
        catch (Exception ignored) {}
    }
    bd.setContentSimilarityBoost(round(similarity));
    if (similarity > 15) reasons.add("内容与你的兴趣高度相似");

    // 热门话题
    double trending = 0;
    try { trending = trendingService.countTrendingTagsInContent(content) * dw.wTrending; }
    catch (Exception ignored) {}
    bd.setTrendingBoost(round(trending));
    if (trending > 0) reasons.add("热门话题");

    // 内容深度匹配
    double depthMatch = computeDepthMatch(content, profile) * dw.wDepthMatch;
    bd.setDepthMatchBoost(round(depthMatch));
    if (depthMatch > 15) reasons.add("符合你的阅读偏好");

    // 新鲜度匹配
    double freshnessVal = computeFreshnessMatch(content, profile) * dw.wFreshness;
    bd.setFreshnessBoost(round(freshnessVal));

    // 协同过滤加成
    double cfBoost = cfRecommendedIds.contains(content.getId()) ? dw.wCollaborative : 0;
    bd.setCollaborativeFilteringBoost(round(cfBoost));
    if (cfBoost > 0) reasons.add("相似用户也喜欢");

    // 时间衰减
    long hoursDiff = 1;
    if (content.getCreatedAt() != null) {
        hoursDiff = Math.max(1, Duration.between(content.getCreatedAt(), LocalDateTime.now()).toHours());
    }
    double timeDecay = calculateTimeDecay(content);
    bd.setTimeDecayFactor(round3(1.0 / timeDecay));
    bd.setHoursAgo(hoursDiff);

    // 探索因子
    double jitter = Math.random() * 5.0;
    bd.setJitter(round(jitter));

    // 最终评分
    double finalScore = (engagement / timeDecay) + topicAffinity + authorAffinity
        + personalization + similarity + trending + depthMatch + freshnessVal + cfBoost + jitter;

    Long cId = content.getId();
    if (dislikedIds.contains(cId)) {
        finalScore *= DISLIKE_PENALTY_FACTOR;
        reasons.clear();
        reasons.add("你已踩过此内容 (×0.02)");
    } else if (likedIds.contains(cId)) {
        finalScore *= LIKE_PENALTY_FACTOR;
        reasons.add("已赞过, 降低优先级 (×0.15)");
    } else if (viewedIds.contains(cId)) {
        finalScore *= VIEWED_PENALTY_FACTOR;
        reasons.add("已浏览过 (×0.5)");
    }

    bd.setFinalScore(round(finalScore));

    // 行为画像信息
    if (reasons.isEmpty()) reasons.add("综合推荐");
    bd.setRecommendReasons(reasons);
    bd.setUserStage(profile.userStage);
    bd.setProfileSummary(profile.profileSummary);
    bd.setDynamicWeights(weightsMap);

    return new ScoredContentWithDetails(content, bd);
}

```

(5) 近期兴趣序列生成逻辑

代码来源: backend/src/main/java/com/example/rec/service/PersonaService.java (第 453-528 行)

```
private Map<String, Object> computeRecentInterestSequence(List<Behavior> allBehaviors, LocalDateTime now) {
    Map<String, Object> result = new LinkedHashMap<>();
    int days = 14;
    LocalDate start = now.toLocalDate().minusDays(days - 1L);

    List<Long> contentIds = allBehaviors.stream()
        .map(Behavior::getContentId)
        .filter(Objects::nonNull)
        .distinct()
        .collect(Collectors.toList());
    Map<Long, Content> contentMap = contentIds.isEmpty()
        ? Collections.emptyMap()
        : contentRepository.findAllById(contentIds).stream()
            .collect(Collectors.toMap(Content::getId, c -> c, (a, b) -> a));

    Map<String, double[]> tagSeries = new HashMap<>();
    List<Map<String, Object>> timeline = new ArrayList<>();
    for (int i = 0; i < days; i++) {
        LocalDate day = start.plusDays(i);
        Map<String, Double> dayScores = new HashMap<>();
        long behaviorCount = 0;

        for (Behavior behavior : allBehaviors) {
            if (behavior.getCreatedAt() == null || !behavior.getCreatedAt().toLocalDate().equals(day)) continue;
            Content content = contentMap.get(behavior.getContentId());
            if (content == null) continue;

            Set<String> labels = extractInterestLabels(content);
            if (labels.isEmpty()) continue;

            behaviorCount++;
            double score = getBehaviorWeight(behavior.getType());
            for (String label : labels) {
                dayScores.merge(label, score, Double::sum);
                tagSeries.computeIfAbsent(label, key -> new double[days])[i] += score;
            }
        }

        String topTag = dayScores.entrySet().stream()
            .max(Map.Entry.comparingByValue())
            .map(Map.Entry::getKey)
            .orElse("暂无");
        double totalScore = dayScores.values().stream().mapToDouble(Double::doubleValue).sum();

        Map<String, Object> item = new LinkedHashMap<>();
        item.put("date", day.format(DAY_FMT));
        item.put("topTag", topTag);
        item.put("behaviorCount", behaviorCount);
        item.put("totalScore", round(totalScore));
        timeline.add(item);
    }

    List<String> labels = new ArrayList<>();
    for (int i = 0; i < days; i++) {
        labels.add(start.plusDays(i).format(DAY_FMT));
    }

    List<Map<String, Object>> series = tagSeries.entrySet().stream()
        .sorted((a, b) -> Double.compare(sum(b.getValue()), sum(a.getValue())))
        .limit(5)
        .map(entry -> {
            Map<String, Object> item = new LinkedHashMap<>();
            item.put("name", entry.getKey());
            item.put("data", Arrays.stream(entry.getValue())
                .map(v -> Math.round(v * 100.0) / 100.0)
                .boxed()
                .collect(Collectors.toList()));
            return item;
        })
        .collect(Collectors.toList());
}
```



```
    result.put("labels", labels);
    result.put("series", series);
    result.put("timeline", timeline);
    return result;
}
```

(6) 作者去重率与标签覆盖率计算逻辑

代码来源: backend/src/main/java/com/example/rec/controller/CompareController.java (第 195-239 行)

```
private Map<String, Object> calculateDiversityStats(List<ContentWithScore> list) {
    Map<String, Object> stats = new HashMap<>();
    if (list == null || list.isEmpty()) {
        stats.put("authorDedupRate", 0.0);
        stats.put("uniqueAuthorCount", 0);
        stats.put("tagCoverageRate", 0.0);
        stats.put("uniqueTagCount", 0);
        return stats;
    }

    Set<Long> authorIds = list.stream()
        .map(ContentWithScore::getContent)
        .filter(Objects::nonNull)
        .map(Content::getAuthor)
        .filter(Objects::nonNull)
        .map(User::getId)
        .filter(Objects::nonNull)
        .collect(Collectors.toSet());

    Set<String> tags = list.stream()
        .map(ContentWithScore::getContent)
        .filter(Objects::nonNull)
        .flatMap(content -> {
            Set<String> names = new LinkedHashSet<>();
            if (content.getTags() != null) {
                content.getTags().stream()
                    .map(tag -> tag.getName())
                    .filter(name -> name != null && !name.isBlank())
                    .forEach(names::add);
            }
            if (names.isEmpty() && content.getCategory() != null && !content.getCategory().isBlank()) {
                names.add(content.getCategory());
            }
            return names.stream();
        })
        .collect(Collectors.toCollection(LinkedHashSet::new));

    double authorDedupRate = (double) authorIds.size() / list.size();
    double tagCoverageRate = Math.min(1.0, (double) tags.size() / list.size());
    stats.put("authorDedupRate", round(authorDedupRate));
    stats.put("uniqueAuthorCount", authorIds.size());
    stats.put("tagCoverageRate", round(tagCoverageRate));
    stats.put("uniqueTagCount", tags.size());
    return stats;
}
```

A.3 对应 4.3 AI 推荐与后台管理模块的核心代码

本模块保留推荐策略切换、AI 推荐降级、AI 打标和数据导入入口等核心实现。

(1) 推荐策略切换逻辑

代码来源：backend/src/main/java/com/example/rec/service/RecommendationStrategyManager.java（第 48-58 行）

```
public void switchStrategy(String type) {
    if (STRATEGY_TRADITIONAL.equals(type) || STRATEGY_AI.equals(type)) {
        activeStrategy.set(type);
        if (STRATEGY_TRADITIONAL.equals(type)) {
            aiRecCacheService.clearAll();
        }
        log.info("[Strategy] Switched to: {}", type);
    } else {
        throw new IllegalArgumentException("Unknown strategy: " + type + ". Use 'traditional' or 'ai'.");
    }
}
```

(2) traditional/ai 统一推荐入口

代码来源：backend/src/main/java/com/example/rec/service/RecommendationStrategyManager.java（第 78-83 行）

```
public List<ContentWithScore> recommendWithScore(Long userId, List<Content> candidates) {
    if (STRATEGY_AI.equals(activeStrategy.get())) {
        return aiRecCacheService.getRecommendationWithScore(userId, candidates);
    }
    return traditionalStrategy.recommendWithScore(userId, candidates);
}
```

(3) AI 推荐排序与异常降级逻辑

代码来源：backend/src/main/java/com/example/rec/service/AiRecommendationStrategy.java（第 90-104 行）

```
public List<ContentWithScore> recommendWithScore(Long userId, List<Content> candidates) {
    if (candidates == null || candidates.isEmpty()) return Collections.emptyList();

    BehaviorProfile profile = userId != null
        ? behaviorProfileService.buildProfile(userId) : new BehaviorProfile();

    List<Content> topCandidates = presortByEngagement(candidates, MAX_CANDIDATES_FOR_AI);

    AiRankingResult aiResult = callAiRankingFull(profile, topCandidates);
    if (aiResult == null) {
        return buildFallbackScored(candidates, profile);
    }

    return buildScoredResult(candidates, topCandidates, aiResult, profile);
}
```

(4) AI 内容打标逻辑

代码来源：backend/src/main/java/com/example/rec/service/AiTaggingService.java（第 55-86 行）

```
public List<String> tagContent(Content content) {
    if (content == null || content.getContent() == null || content.getContent().isBlank()) {
        return Collections.emptyList();
    }

    // 1. 获取所有候选标签
    List<String> candidateTags = tagRepository.findAll().stream()
        .map(Tag::getName)
        .collect(Collectors.toList());

    // 2. 构造 Prompt
    String prompt = buildPrompt(content.getContent(), candidateTags);

    // 3. 调用 Ollama API
    String aiResponse = callOllama(prompt);
    if (aiResponse == null || aiResponse.isBlank()) {
```

```

        return Collections.emptyList();
    }

    // 4. 解析标签
    List<String> tagNames = parseTagResponse(aiResponse, candidateTags);

    // 5. 写入数据库
    for (String tagName : tagNames) {
        Tag tag = tagRepository.findByName(tagName)
            .orElseGet(() -> tagRepository.save(new Tag(tagName)));
        content.getTags().add(tag);
    }
    contentRepository.save(content);

    return tagNames;
}

```

(5) 后台推荐策略切换接口

代码来源: backend/src/main/java/com/example/rec/controller/AdminController.java (第 58-70 行)

```

@PostMapping("/rec-strategy")
public Map<String, Object> switchRecStrategy(@RequestBody Map<String, String> payload) {
    String strategy = payload.get("strategy");
    if (strategy == null || strategy.isBlank()) {
        return Map.of("error", "Missing 'strategy' field");
    }
    try {
        strategyManager.switchStrategy(strategy);
        return Map.of("message", "策略已切换", "current", strategyManager.getCurrentStrategyType());
    } catch (IllegalArgumentException e) {
        return Map.of("error", e.getMessage());
    }
}

```

(6) 后台批量导入数据接口

代码来源: backend/src/main/java/com/example/rec/controller/AdminController.java (第 75-92 行)

```

@PostMapping("/import-kaggle-batch")
public Map<String, Object> importKaggleBatch(@RequestBody(required = false) Map<String, Object> payload) {
    int target = 50;
    if (payload != null && payload.get("target") != null) {
        target = ((Number) payload.get("target")).intValue();
    }
    KaggleImportService.ImportResult result = kaggleImportService.batchImport(target);

    Map<String, Object> resp = new HashMap<>();
    resp.put("success", result.success);
    resp.put("message", result.message);
    resp.put("importedCount", result.importedCount);
    resp.put("skippedDuplicate", result.skippedDuplicate);
    resp.put("source", result.source);
    resp.put("startTime", result.startTime != null ? result.startTime.toString() : null);
    resp.put("endTime", result.endTime != null ? result.endTime.toString() : null);
    return resp;
}

```

A.4 对应 4.4 广告分发模块的核心代码

本模块保留广告画像匹配、广告曝光点击记录和前端广告事件上报代码。

(1) 基于画像标签的广告匹配逻辑

代码来源：backend/src/main/java/com/example/rec/service/AdService.java（第 81-133 行）

```
@SuppressWarnings("unchecked")
public List<Map<String, Object>> getRelevantAds(Long userId, int count) {
    // 1. 获取用户兴趣标签
    Map<String, Object> persona = personaService.getUserPersona(userId);
    List<String> userTags = (List<String>) persona.getDefault("interestTags", Collections.emptyList());

    // 2. 获取活跃广告
    List<Ad> activeAds = adRepository.findByActiveTrue();
    if (activeAds.isEmpty()) return Collections.emptyList();

    // 3. 计算得分并排序
    Map<Long, Integer> userFreq = freqCapMap.getDefault(userId, Collections.emptyMap());

    List<Map<String, Object>> scoredAds = new ArrayList<>();
    for (Ad ad : activeAds) {
        // 频控检查
        int shown = userFreq.getDefault(ad.getId(), 0);
        if (shown >= MAX_IMPRESSIONS_PER_USER) continue;

        // 标签匹配
        List<String> matchedTags = new ArrayList<>();
        if (ad.getTargetTags() != null && !userTags.isEmpty()) {
            String[] adTags = ad.getTargetTags().split(",");
            for (String adTag : adTags) {
                String trimmed = adTag.trim();
                if (userTags.contains(trimmed)) {
                    matchedTags.add(trimmed);
                }
            }
        }

        // 质量分（历史 CTR）
        double ctr = ad.getImpressionCount() > 0 ?
            (double) ad.getClickCount() / ad.getImpressionCount() : 0.05; // 新广告默认 5%
        double qualityScore = ctr * 100;

        // 综合得分
        double adScore = matchedTags.size() * 30
            + (ad.getBidPrice() != null ? ad.getBidPrice() : 1.0) * 10
            + qualityScore;

        Map<String, Object> item = new HashMap<>();
        item.put("ad", ad);
        item.put("score", Math.round(adScore * 100.0) / 100.0);
        item.put("matchedTags", matchedTags);
        item.put("ctr", Math.round(ctr * 10000.0) / 100.0); // 百分比保留 2 位
        scoredAds.add(item);
    }

    // 按得分降序
    scoredAds.sort((a, b) -> Double.compare((Double) b.get("score"), (Double) a.get("score")));
    return scoredAds.stream().limit(count).collect(Collectors.toList());
}
```

(2) 广告曝光记录逻辑

代码来源：backend/src/main/java/com/example/rec/service/AdService.java（第 138-149 行）

```
public void recordImpression(Long adId, Long userId) {
    adRepository.findById(adId).ifPresent(ad -> {
        ad.setImpressionCount(ad.getImpressionCount() + 1);
        adRepository.save(ad);
    });

    // 更新频控计数
```

```
        if (userId != null) {
            freqCapMap.computeIfAbsent(userId, k -> new ConcurrentHashMap<>())
                .merge(adId, 1, Integer::sum);
        }
    }
}
```

(3) 广告点击记录逻辑

代码来源: backend/src/main/java/com/example/rec/service/AdService.java (第 154-159 行)

```
public void recordClick(Long adId) {
    adRepository.findById(adId).ifPresent(ad -> {
        ad.setClickCount(ad.getClickCount() + 1);
        adRepository.save(ad);
    });
}
```

(4) 前端广告曝光与点击上报逻辑

代码来源: frontend/src/components/AdCard.vue (第 50-69 行)

```
const advertiserInitial = computed(() => {
    return props.ad.advertiser ? props.ad.advertiser.charAt(0).toUpperCase() : 'A'
})
```

```
// 记录展示
onMounted(() => {
    if (props.ad.id) {
        api.post(`/ads/${props.ad.id}/impression`, null, {
            params: { userId: props.userId }
        }).catch(() => {})
    }
})
```

```
// 记录点击
const handleClick = () => {
    if (props.ad.id) {
        api.post(`/ads/${props.ad.id}/click`).catch(() => {})
    }
    if (props.ad.targetUrl) {
        window.open(props.ad.targetUrl, '_blank')
    }
}
```

A.5 对应 4.5 数据统计与画像可视化模块的核心代码

本模块保留行为类型分布、内容标签分布、近期兴趣序列展示和统计图表渲染相关代码。

(1) 行为类型分布统计逻辑

代码来源: backend/src/main/java/com/example/rec/service/AnalyticsService.java (第 141-193 行)

```
private Map<String, Object> buildBehaviorStats(List<Behavior> behaviors, int trendDays) {
    Map<String, Long> typeCounts = new LinkedHashMap<>();
    for (String type : BEHAVIOR_TYPES) {
        typeCounts.put(type, 0L);
    }
    for (Behavior b : behaviors) {
        if (b.getType() == null) continue;
        String type = b.getType().toUpperCase();
        typeCounts.merge(type, 1L, Long::sum);
    }

    List<Map<String, Object>> typeDistribution = typeCounts.entrySet().stream()
        .map(e -> metric(e.getKey(), behaviorLabel(e.getKey()), e.getValue()))
        .toList();

    int days = Math.max(1, trendDays);
    LocalDate start = LocalDate.now().minusDays(days - 1L);
    List<Map<String, Object>> dailyTrend = new ArrayList<>();
    for (int i = 0; i < days; i++) {
        LocalDate day = start.plusDays(i);
        Map<String, Object> item = new LinkedHashMap<>();
        item.put("date", day.format(DAY_FMT));
        long total = 0;
        for (String type : BEHAVIOR_TYPES) {
            long count = behaviors.stream()
                .filter(b -> type.equalsIgnoreCase(b.getType()))
                .filter(b -> b.getCreatedAt() != null && b.getCreatedAt().toLocalDate().equals(day))
                .count();
            item.put(type, count);
            total += count;
        }
        item.put("total", total);
        dailyTrend.add(item);
    }

    List<Map<String, Object>> hourlyDistribution = new ArrayList<>();
    for (int hour = 0; hour < 24; hour++) {
        int h = hour;
        long count = behaviors.stream()
            .filter(b -> b.getCreatedAt() != null && b.getCreatedAt().getHour() == h)
            .count();
        Map<String, Object> item = new LinkedHashMap<>();
        item.put("hour", hour);
        item.put("count", count);
        hourlyDistribution.add(item);
    }

    Map<String, Object> stats = new LinkedHashMap<>();
    stats.put("typeDistribution", typeDistribution);
    stats.put("dailyTrend", dailyTrend);
    stats.put("hourlyDistribution", hourlyDistribution);
    return stats;
}
```

(2) 内容分类与标签分布统计逻辑

代码来源: backend/src/main/java/com/example/rec/service/AnalyticsService.java (第 244-310 行)

```
private Map<String, Object> buildContentStats(List<Content> contents, int trendDays, int trendingHours) {
    List<Content> topLevelPosts = contents.stream()
        .filter(c -> c.getParentContent() == null)
        .toList();

    Map<String, Long> categories = topLevelPosts.stream()
        .collect(Collectors.groupingBy(
            c -> normalizeBlank(c.getCategory(), "未分类"),
            LinkedHashMap::new,
            Collectors.counting()
        ));
}
```

```

    ));

    Map<String, Long> tagCounts = new HashMap<>();
    for (Content content : topLevelPosts) {
        if (content.getTags() == null || content.getTags().isEmpty()) continue;
        content.getTags().stream()
            .map(tag -> tag.getName())
            .filter(name -> name != null && !name.isBlank())
            .forEach(name -> tagCounts.merge(name, 1L, Long::sum));
    }

    int days = Math.max(1, trendDays);
    LocalDate start = LocalDate.now().minusDays(days - 1L);
    List<Map<String, Object>> dailyPostTrend = new ArrayList<>();
    for (int i = 0; i < days; i++) {
        LocalDate day = start.plusDays(i);
        long count = topLevelPosts.stream()
            .filter(c -> c.getCreatedAt() != null && c.getCreatedAt().toLocalDate().equals(day))
            .count();
        Map<String, Object> item = new LinkedHashMap<>();
        item.put("date", day.format(DAY_FMT));
        item.put("count", count);
        dailyPostTrend.add(item);
    }

    long withImage = topLevelPosts.stream()
        .filter(c -> c.getImageUrl() != null && !c.getImageUrl().isBlank())
        .count();
    double imageRatio = topLevelPosts.isEmpty() ? 0 : round((double) withImage / topLevelPosts.size() * 100);

    List<Map<String, Object>> topPosts = topLevelPosts.stream()
        .sorted(Comparator.comparingLong(this::engagementScore).reversed())
        .limit(10)
        .map(this::toTopPost)
        .toList();

    List<TrendingTopic> trending = trendingService.getTrendingTopics(10, trendingHours);
    List<Map<String, Object>> trendingTopics = trending.stream()
        .map(t -> {
            Map<String, Object> item = new LinkedHashMap<>();
            item.put("name", t.getName());
            item.put("postCount", t.getPostCount());
            item.put("engagement", t.getEngagement());
            item.put("score", round(t.getScore()));
            return item;
        })
        .toList();

    Map<String, Object> stats = new LinkedHashMap<>();
    stats.put("categoryDistribution", toNameValueList(categories));
    stats.put("tagDistribution", toSortedNameValueList(tagCounts, 12));
    stats.put("dailyPostTrend", dailyPostTrend);
    stats.put("topPosts", topPosts);
    stats.put("imagePostRatio", imageRatio);
    stats.put("trendingTopics", trendingTopics);
    return stats;
}

```

(3) 统计中心行为分布与标签分布图表渲染逻辑

代码来源: frontend/src/views/AnalyticsView.vue (第 400-480 行)

```

}

const initChart = (elRef, option) => {
    if (!elRef.value) return
    const chart = echarts.init(elRef.value)
    chart.setOption(option)
    charts.push(chart)
}

const renderCharts = () => {
    disposeCharts()

    const behaviorDistribution = behaviorStats.value.typeDistribution || []
    const dailyTrend = behaviorStats.value.dailyTrend || []
    const dailyLabels = dailyTrend.map(item => item.date)

```

```

initChart(behaviorTrendChart, {
  tooltip: { trigger: 'axis' },
  legend: { top: 0, data: ['浏览', '点赞', '评论', '点踩', '搜索'] },
  grid: { left: 36, right: 20, top: 48, bottom: 28 },
  xAxis: { type: 'category', data: dailyLabels },
  yAxis: { type: 'value' },
  series: [
    lineSeries('浏览', dailyTrend.map(i => i.VIEW || 0), '#1d9bf0'),
    lineSeries('点赞', dailyTrend.map(i => i.LIKE || 0), '#f91880'),
    lineSeries('评论', dailyTrend.map(i => i.COMMENT || 0), '#00ba7c'),
    lineSeries('点踩', dailyTrend.map(i => i.DISLIKE || 0), '#f4212e'),
    lineSeries('搜索', dailyTrend.map(i => i.SEARCH || 0), '#7856ff')
  ]
})

initChart(behaviorPieChart, {
  tooltip: { trigger: 'item' },
  legend: { bottom: 0 },
  series: [{
    type: 'pie',
    radius: ['42%', '70%'],
    center: ['50%', '45%'],
    avoidLabelOverlap: true,
    data: behaviorDistribution
      .filter(item => Number(item.value || item.count || 0) > 0)
      .map(item => ({ name: item.label, value: item.count }))
  ]
})

initChart(stageChart, pieOption(userStats.value.stageDistribution || [], ['#1d9bf0', '#ffad1f', '#00ba7c']))
initChart(userTypeChart, pieOption(userStats.value.typeDistribution || [], ['#1d9bf0', '#7856ff', '#00ba7c']))

initChart(categoryChart, {
  tooltip: { trigger: 'axis' },
  grid: { left: 36, right: 20, top: 26, bottom: 36 },
  xAxis: { type: 'category', data: (contentStats.value.categoryDistribution || []).map(i => i.name) },
  yAxis: { type: 'value' },
  series: [{
    type: 'bar',
    data: (contentStats.value.categoryDistribution || []).map(i => i.value),
    itemStyle: { color: '#1d9bf0', borderRadius: [4, 4, 0, 0] },
    barMaxWidth: 42
  }]
})

const tagDistribution = contentStats.value.tagDistribution || []
initChart(tagChart, {
  tooltip: { trigger: 'axis' },
  grid: { left: 36, right: 20, top: 26, bottom: 54 },
  xAxis: {
    type: 'category',
    data: tagDistribution.map(i => i.name),
    axisLabel: { interval: 0, rotate: 30, fontSize: 11 }
  },
  yAxis: { type: 'value' },
  series: [{
    type: 'bar',
    data: tagDistribution.map(i => i.value),
    itemStyle: { color: '#7856ff', borderRadius: [4, 4, 0, 0] },
    barMaxWidth: 42
  }]
})

const postTrend = contentStats.value.dailyPostTrend || []
initChart(postTrendChart, {

```

(4) 近期兴趣序列页面展示模板

代码来源: frontend/src/components/PersonaDetailCard.vue (第 55-80 行)

```
</div>
```

```
<!-- 近期兴趣序列 -->
```

```
<div class="section recent-interest-section" v-if="hasRecentInterestSequence">
  <h4>近期兴趣序列</h4>
```



```

<div ref="recentInterestChart" class="recent-interest-chart"></div>
<div class="recent-timeline">
  <div
    class="timeline-item"
    v-for="item in persona.recentInterestSequence.timeline"
    :key="item.date"
    :class="{ muted: !item.behaviorCount }"
  >
    <span class="timeline-date">{{ item.date }}</span>
    <span class="timeline-tag">{{ item.topTag }}</span>
  </div>
</div>
</div>

<!-- 活跃时段柱状图 -->
<div class="section time-section" v-if="persona.hourlyDistribution">
  <h4>🕒 活跃时段分布</h4>
  <div ref="timeBarChart" class="time-bar-chart"></div>
  <div class="night-owl" v-if="persona.nightOwlIndex !== undefined">
    🦉 夜猫子指数: <strong>{{ Math.round(persona.nightOwlIndex * 100) }}%</strong>
  </div>
</div>

```

(5) 近期兴趣序列折线图数据构造逻辑

代码来源: frontend/src/components/PersonaDetailCard.vue (第 190-220 行)

```

    })
  })
}

const renderRecentInterest = () => {
  const sequence = props.persona?.recentInterestSequence
  if (!recentInterestChart.value || !sequence?.labels?.length) return
  if (!recentInterestChartInstance) {
    recentInterestChartInstance = echarts.init(recentInterestChart.value)
  }

  const palette = ['#1DA1F2', '#00BA7C', '#FFAD1F', '#7856FF', '#F91880']
  const series = (sequence.series || []).map((item, index) => ({
    name: item.name,
    type: 'line',
    smooth: true,
    symbolSize: 6,
    data: item.data || [],
    lineStyle: { width: 3 },
    itemStyle: { color: palette[index % palette.length] },
    areaStyle: { opacity: 0.08 }
  })))

  recentInterestChartInstance.setOption({
    tooltip: { trigger: 'axis' },
    legend: { top: 0, type: 'scroll' },
    grid: { left: 36, right: 18, top: 48, bottom: 28 },
    xAxis: {
      type: 'category',
      data: sequence.labels,
      axisLabel: { fontSize: 11, color: '#536471' },

```